AKADEMIA GÓRNICZO-HUTNICZA IM. STANISŁAWA STASZICA W KRAKOWIE

Faculty of Electrical Engineering, Automatics, Computer Science and Biomedical Engineering

Engineering Thesis

Enhancing Small Language Models via Chain-of-Thought Distillation

Ulepszanie małych modeli językowych poprzez destylację łańcucha myślowego

Author: *Kacper Rabczewski*
Degree programme: *Computer science and intelligent systems*
Supervisor: *Dr. Krzysztof Kluza*

Kraków, 2025

Put your acknowledgements here

Abstract

This thesis investigates whether supervised chain-of-thought distillation (SCoTD) can measurably improve the reasoning accuracy of a small language model on math word problems. A strong teacher model generates structured reasoning traces (CoT) for GSM8K, which are cleaned and deduplicated, and then used to finetune a 3B-class student via parameter-efficient adapters (LoRA) under 4-bit quantization. The training stack relies on modern open tooling for reproducibility and efficient use of a single 24GB GPU. Evaluation uses strict numeric exact-match parsing on GSM8K.

Results show that SCoTD improves the student over a non-finetuned base with CoT prompting (78.54% vs 70.51%). A label-only variant also improves over its baseline (15.01% vs 10.99%), though strict formatting requirements likely undercount true ability. The teacher reaches 93.61% on the GSM8K test split. Contributions include a cost-aware CoT data generation pipeline, a reproducible finetuning setup (LoRA + 4-bit nf4 + bf16), and an empirical analysis of accuracy, training dynamics, and operational costs. These findings support SCoTD as a practical method to enhance small models' reasoning under resource constraints.

Keywords: chain-of-thought, distillation, LoRA, QLoRA, GSM8K, small language models, reasoning, quantization

Contents

1. Introduction

Large language models (LLMs) have achieved strong performance on a wide range of language tasks. However, small models (sub-4B parameters) often fall short on multi-step reasoning benchmarks such as GSM8K, where solutions require decomposing a problem into intermediate steps and computing a precise numeric answer [1]. Chain-of-thought (CoT) prompting can elicit step-by-step reasoning from strong teachers and improve accuracy [2], especially when combined with self-consistency sampling [3]. Yet, relying on large models at inference time is costly and slow. This motivates distilling teacher reasoning traces into a small student that can run efficiently on commodity hardware.

This thesis studies supervised chain-of-thought distillation (SCoTD): a strong teacher generates structured CoTs for GSM8K questions; a small student is finetuned to reproduce these traces and final answers. The student is trained with parameter-efficient finetuning (LoRA) [4] on quantized weights (QLoRA) [5], enabling training on a single 24GB GPU while preserving headroom to learn reasoning behavior. The approach uses standard open-source tooling for models and tokenization [6, 7, 8].

Problem and motivation

Small models are attractive due to their lower latency, footprint, and cost, but they struggle with multi-step math reasoning under greedy decoding. CoT distillation promises to transfer the teacher's intermediate reasoning patterns, potentially improving student accuracy without increasing inference-time cost. The practical question is whether a 3B-class model can benefit sufficiently from SCoTD to close part of the gap to teacher performance on GSM8K.

Research questions

– RQ1: Does SCoTD improve a 3B-class model on GSM8K compared to the non-finetuned base under CoT prompting?

– RQ2: How does label-only supervised finetuning compare to SCoTD and base baselines under strict numeric exact-match evaluation?

Scope

The study focuses on a single small decoder-only model (Qwen2.5-3B) [9] and a single dataset (GSM8K) [1]. Teacher CoTs are produced via an API-accessible, strong reasoning model configured to emit structured steps and a line-form final answer. The student is trained with LoRA on 4-bit quantized base weights using bf16 compute, gradient checkpointing, and standard optimizers. Evaluation uses greedy decoding with strict numeric parsing; outputs that deviate from the expected format are counted as incorrect.

Contributions

– A practical, cost-aware pipeline for generating and cleaning teacher CoTs for GSM8K, with concurrency-tuned API calls and per-sample metadata for analysis.

– A reproducible PEFT training setup (LoRA + 4-bit nf4 + bf16) that fits on a single 24GB GPU while targeting attention and MLP projection modules.

– Empirical evidence that SCoTD improves a 3B-class model over base CoT prompting on GSM8K (78.54% vs 70.51%), with a strong teacher at 93.61% and a label-only student that improves over baseline (15.01% vs 10.99%) despite strict-format penalties.

– Operational notes on prompt design, cleaning/deduplication, truncation policy, and evaluation pitfalls that affect measured accuracy.

Method overview

The pipeline comprises teacher CoT generation, cleaning and deduplication, dataset processing, student finetuning (SCoTD and label-only variants), and evaluation. The student relies on PEFT with LoRA adapters [4] over a quantized backbone [5]. Tokenization is handled with a fast subword tokenizer [7, 8]. Inference uses greedy decoding and a strict numeric exact-match metric. The software stack builds on Transformers and related tooling [6]. The teacher is a strong reasoning model (DeepSeek-R1 Distill [10]) accessed via OpenRouter [11].

Results summary

SCoTD yields a clear gain over the base model under CoT prompting (78.54% vs 70.51%). Label-only finetuning improves over its baseline (15.01% vs 10.99%), though strict formatting likely undercounts correct numeric computations. The teacher achieves 93.61% on GSM8K test, providing headroom and confirming the value of distilling high-quality reasoning traces.

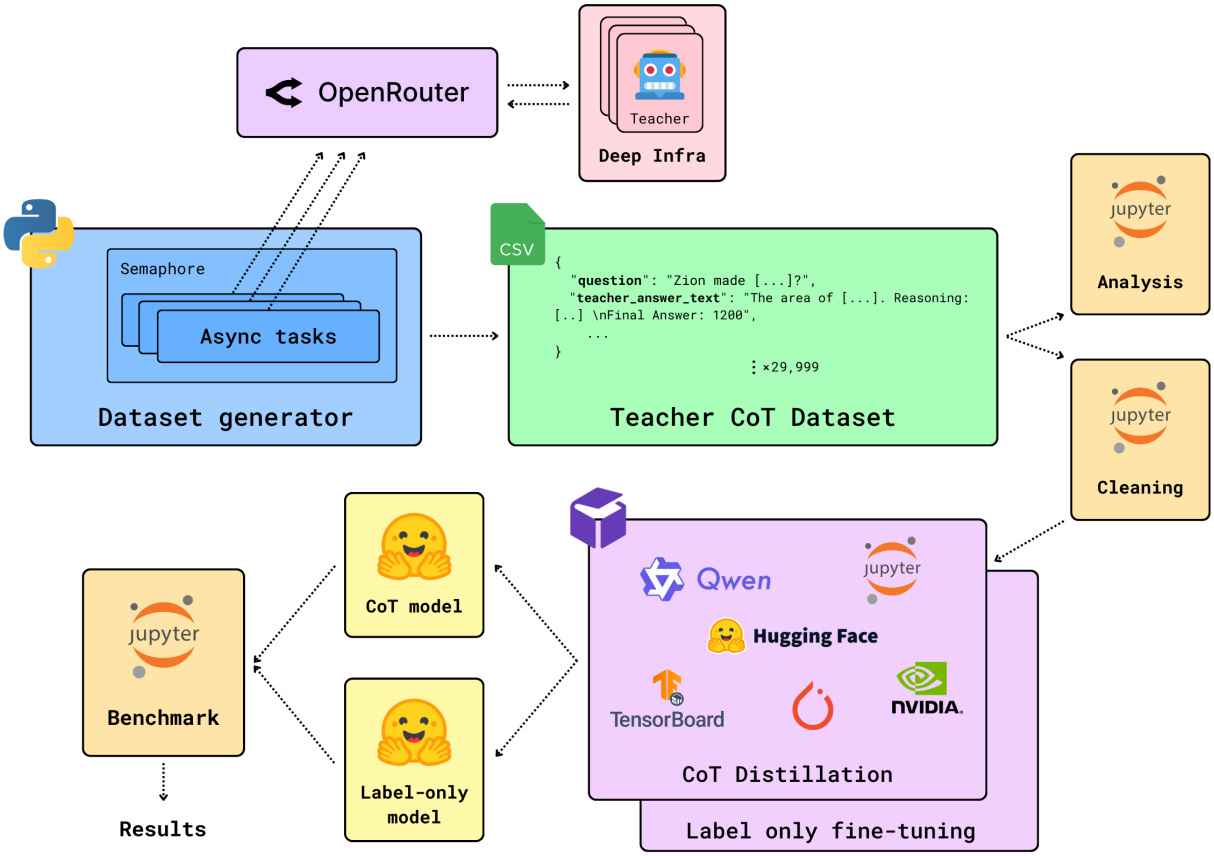


Figure 1.1. End-to-end SCoTD pipeline.

Author background and motivation

The author is a Co-Founder at an AI phishing detection startup that uses LLMs to analyze URLs and webpage content for malicious indicators, and a Machine Learning Engineer at OneRail building LLM-powered support workflows for same-day logistics. These roles highlighted the need for reliable, cost-efficient reasoning under tight latency and single-GPU constraints, motivating SCoTD as a practical path to higher accuracy.

2. Background

2.1. Large language models

Large language models (LLMs) lie at the core of the current AI revolution. They are parametric probabilistic models that learn to represent the distribution of natural language sequences and generate text by autoregressively predicting one token at a time. Formally, given a tokenized sequence $(x_1, \ldots, x_T)$, an LLM models $p(x_{1:T}) = \prod_{t=1}^{T} p(x_t \mid x_{<t})$ and is trained to minimize the cross-entropy between predicted and observed tokens. LLMs derive their broad linguistic and factual knowledge from pretraining on massive text corpora using this objective, which at scale enables emergent capabilities such as in-context learning and reasoning.

2.1.1. Objective and pretraining

The standard objective is a next-token prediction:

$$\mathcal{L}(\theta) = -\sum_{t=1}^{T} \log p_\theta(x_t \mid x_{<t}), \tag{2.1}$$

optimized over large, diverse datasets using gradient descent. Conceptually, this process can be understood as a vast, GPU-driven lossy compression - the model distills the statistical regularities and patterns of human language into a compact, parameterized form that enables general-purpose text understanding and generation. While pretraining is purely self-supervised, downstream adaptation may use supervised fine-tuning or preference-based methods to align behavior.

2.1.2. Transformer

Transformer architecture [12] underlies most modern LLMs. The main building blocks are:

– **Embeddings and positional encoding**: Tokens are mapped to dense vectors and augmented with position information, either sinusoidal or learned, to make the model order-aware.

– **Multi-head self-attention**: For each position, queries Q, keys K, and values V are computed by linear projections of the hidden states. Scaled dot-product attention aggregates information across positions:

$$\mathrm{Attn}(Q, K, V) = \mathrm{softmax}\left(\frac{QK^\top}{\sqrt{d_k}}\right) V. \tag{2.2}$$

Multiple heads attend in parallel to different subspaces, and their outputs are concatenated and projected.

– **Position-wise feed-forward network (FFN)**: A two-layer MLP with a nonlinearity (e.g., GELU) is applied independently to each position.

– **Residual connections and layer normalization**: Each sublayer is wrapped with residual additions and normalization for stability.

Computationally, attention has $O(L^2 d)$ time and $O(L^2)$ memory in sequence length L (per batch and head factors omitted), while FFNs dominate parameter count with $O(d^2)$.

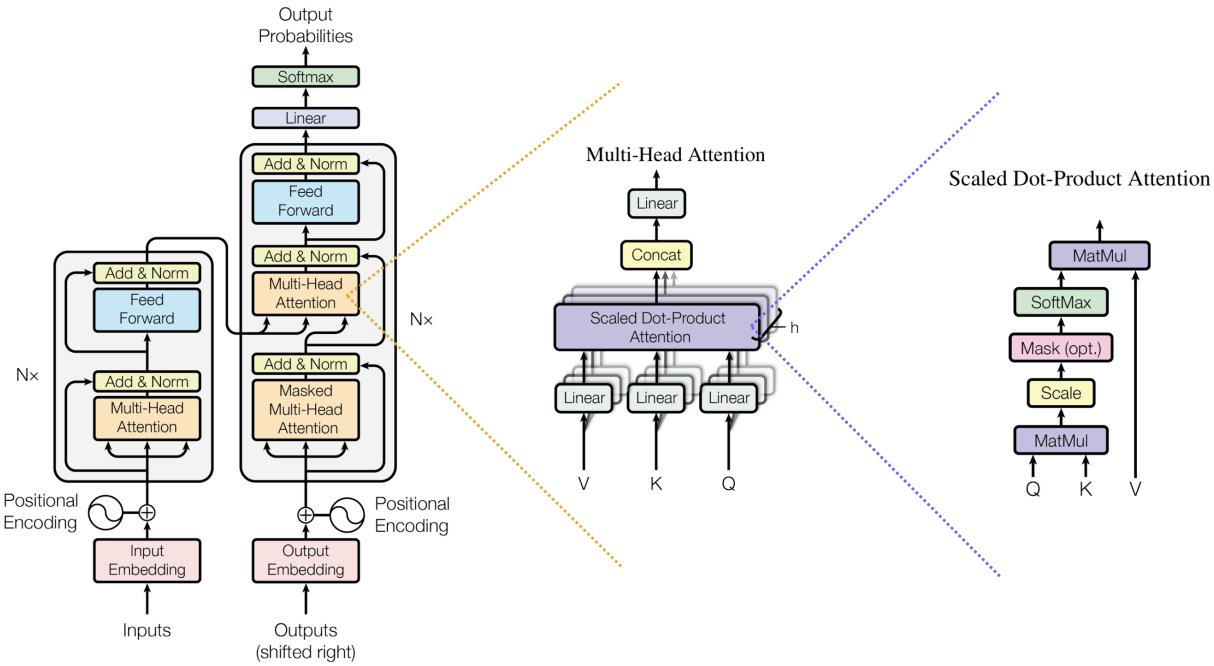


Figure 2.1. Transformer encoder-decoder block [12].

2.1.3. Decoder-only LLMs

Decoder-only LMs use a lower-triangular mask in self-attention so that token t attends only to positions $< t$. This enforces autoregressive factorization and enables left-to-right generation. During inference, key-value (KV) caching stores past projections to avoid recomputation, reducing per-token latency from quadratic to approximately linear in L for attention, with constant-depth projections per layer.

2.1.4. Scaling laws

Transformers scale predictably with model size (parameters), data (tokens), and compute. Empirically, test loss follows power-law scaling with these resources [13], implying steady gains as capacity and data increase. Practical costs rise superlinearly with size due to:

– **Training**: attention's $O(L^2)$ complexity amplifies FLOPs and memory.

– **Inference**: latency and memory scale with depth and width; context length increases multiply attention cost without specialized kernels.

Despite costs, scaling yields emergent capabilities such as in-context learning and improved generalization [14].

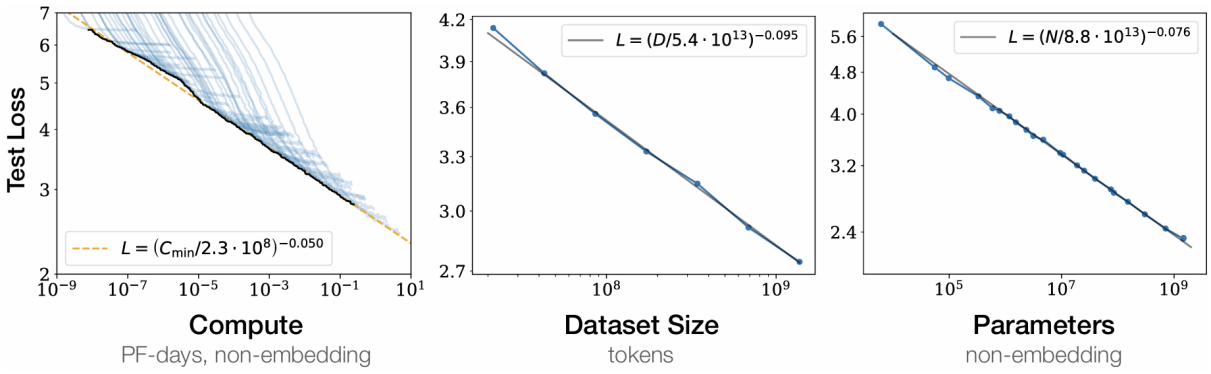


Figure 2.2. Scaling laws: test loss vs. effective compute/model size/data on logarithmic axes [13].

2.1.5. Limitations and small-model challenges

Large language models are known to have several limitations including:

– **Hallucinations**: the model may produce fluent but unsupported statements.

– **Context and computation limits**: finite context windows and lack of external working memory hinder long-horizon tasks.

– **Compositionality**: chaining steps and maintaining intermediate variables is difficult under greedy decoding.

Small models (e.g., <4B parameters) face additional constraints which degrade multi-step arithmetic and logical reasoning, especially when strict output formats are required. While prompting and decoding strategies can mitigate some issues, structural capacity often remains the bottleneck without additional supervision or architectural support.

2.2. Prompting

Prompting is a practice of specifying a model's task through natural language. It functions as a programming interface to language models. Instruction tuning and alignment methods improve adherence to prompts [15].

2.2.1. Paradigms

Several prompting paradigms have emerged, often combined in practice:

– **Zero-shot prompting**: The model receives only task instructions and the input. This tests inherent generalization from pretraining and any instruction tuning. It is simple and cheap but sensitive to wording and often weaker on tasks requiring intermediate computation.

– **Few-shot prompting**: The prompt includes k input-output examples that implicitly define the task and desired format, enabling in-context learning [14].

– **Chain-of-Thought (CoT) prompting**: The prompt requests step-by-step reasoning, eliciting intermediate rationales before the final answer [2]. CoT can improve complex, multi-step tasks like arithmetic by encouraging deliberate decomposition, but increases latency and token usage.

2.3. Knowledge distillation

Knowledge distillation (KD) transfers the behavior of a high-capacity teacher model to a typically smaller student [16]. The core idea is that softened target distributions carry "dark knowledge" about class or token similarities that hard labels do not expose, often improving generalization while enabling compression and faster inference.

2.3.1. Objectives and loss design

Response-based KD is the most widely used approach. In this setting, the student is trained to match the teacher's output distribution. Let $z^{(T)}$ and $z^{(S)}$ be teacher and student logits, and $T > 0$ a temperature. The softened distributions are $p^{(T)} = \mathrm{softmax}(z^{(T)}/T)$ and $p^{(S)} = \mathrm{softmax}(z^{(S)}/T)$. A common loss combines distillation with standard supervision:

$$\mathcal{L} = (1-\alpha)\,\mathrm{CE}(y,\ \mathrm{softmax}(z^{(S)})) + \alpha\, T^2\, \mathrm{KL}\Big(p^{(T)} \,\|\, p^{(S)}\Big)\,, \tag{2.3}$$

where y are hard labels and $\alpha \in [0,1]$ balances terms; the T^2 factor compensates for gradient scaling [16].

Beyond responses, feature-based KD aligns intermediate representations, while relation-based KD matches instance relations. Self-distillation trains a student from its own or a previous generation's predictions (Born-Again Networks [17]); data-augmented KD uses pseudo-labels on unlabeled data (Noisy Student [18]). See [19] for a survey.

2.3.2. Distillation for sequence models and LLMs

In language modeling, KD is applied at either:

– **Token level**: match teacher logits or probabilities at each decoding step, often alongside cross-entropy to the ground truth.

– **Sequence level**: train on sequences decoded by the teacher (e.g., beam search), simplifying the target distribution and reducing mode entropy [20].

Both offline (fixed teacher) and online (co-trained or periodically refreshed teacher) regimes exist. For autoregressive models, exposure bias remains; KD reduces variance but does not remove the mismatch between training and generation. Practical choices include which layers to distill, how to weight losses over time, and whether to distill only on high-confidence tokens.

2.3.3. Distilling reasoning

Symbolic Chain-of-Thought Distillation (SCoTD) transfers step-by-step reasoning from a large teacher to a smaller student by training on teacher-generated rationales rather than logits [21]. Concretely: (i) curate a few-shot CoT prompt set; (ii) for each unlabeled input x, sample multiple CoT rationales z and a prediction y from the teacher; (iii) optionally filter samples to keep only label-correct pairs when gold labels are available; and (iv) fine-tune the student with the standard token-by-token language modeling objective to generate the concatenated rationale and answer. At inference, the student can decode a single rationale+answer greedily or sample multiple reasoning paths and select the majority answer via self-consistency [3].

Empirically, SCoTD enables 125M-1.3B students to "think step-by-step"and improves accuracy on commonsense QA benchmarks (CSQA, QuaRel, OpenBookQA). A key finding is that oversampling rationales per input (e.g., $\sim$30 CoTs/example) is more important than aggressive filtering. Self-consistency particularly helps when training used noisy, unfiltered CoTs (few-shot setting), whereas its benefit diminishes once supervised filtering removes incorrect teacher answers.

2.3.4. When to expect gains

KD is most effective when the teacher's distribution contains informative structure that the student cannot infer from hard labels alone. For CoT, expected gains increase with teacher rationale quality and student capacity to model longer sequences.

2.4. Parameter-efficient fine-tuning (PEFT)

Full fine-tuning updates all model parameters and optimizer states, which scales memory, compute, and communication roughly linearly with parameter count and often requires multi-GPU setups. Optimizers like Adam maintain first and second moments, tripling memory relative to parameters alone. Checkpointing, gradient storage, and activation memory further increase requirements. Parameter-efficient fine-tuning (PEFT) alleviates these costs by freezing the backbone and training a small number of additional parameters that modulate the forward pass. This also improves modularity (task-specific heads), reduces overfitting, and enables fast switching between tasks.

2.4.1. Method families

Several PEFT families differ in where trainable parameters are introduced and how they influence the network:

- **Adapters**: Small bottleneck modules (down-projection, nonlinearity, up-projection) are inserted within Transformer blocks and trained while the base model is frozen [22]. They achieve near full fine-tuning performance with orders of magnitude fewer trainable parameters and support composition across tasks.

- **Prompt and prefix tuning**: Instead of modifying internal weights, these methods learn task-specific soft prompts. Prompt tuning optimizes a small set of embeddings prepended to the input [23], while prefix-tuning optimizes continuous key/value prefixes injected into attention at each layer [24]. These approaches are extremely parameter-efficient and work well when the pretrained model already encodes the needed capabilities; performance can depend on model scale and task complexity.

- **Low-rank adaptation (LoRA)**: LoRA freezes the original weight matrix W and parameterizes the update as a low-rank decomposition $\Delta W = AB$ with rank $r \ll \min(d_{\text{in}}, d_{\text{out}})$ [4]. A common scaled form is

$$W' = W + \frac{\alpha}{r} AB, \tag{2.4}$$

  where $A \in \mathbb{R}^{d_{\text{out}} \times r}$ and $B \in \mathbb{R}^{r \times d_{\text{in}}}$ are the only trainable parameters and α controls the update magnitude. LoRA is typically applied to attention and MLP projection matrices. Benefits include: (i) minimal trainable parameters, (ii) negligible inference overhead when merged into W, and (iii) modular adapters that can be stored and shared independently of the base model.

2.4.2. Quantization-aware PEFT

Quantization-aware fine-tuning combines PEFT with low-precision base weights to further reduce memory. QLoRA keeps the frozen backbone in 4-bit quantization (e.g., NF4) with per-channel scaling and "double quantization" to compress scales, while performing forward/backward compute in higher precision (e.g., bf16) and training only the PEFT parameters [5]. In this setup:

- The quantized backbone reduces memory footprint without updating full-precision weights.

- LoRA (or other PEFT parameters) are maintained in higher precision and receive gradients.

- Careful quantizer design (NF4) and optimizer choices preserve training stability and accuracy close to full-precision baselines.

This approach enables fine-tuning larger backbones under tight memory budgets and decouples adaptation capacity (set by PEFT parameters like LoRA rank) from the storage format of the base model. Practical backends for 4-bit inference/training are widely available [25].

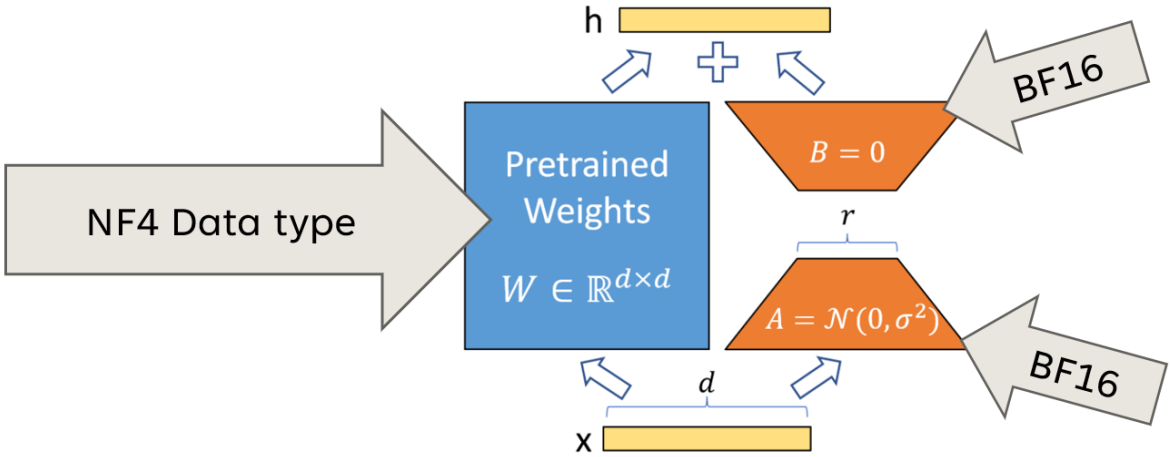


Figure 2.3. Model parameters under QLoRA NF4 + LoRA: frozen backbone in 4-bit quantization with per-channel scales; trainable LoRA parameters in higher precision. Forward/backward compute in bf16 [26].

2.5. Tokenizers

Tokenization converts raw text into a sequence of discrete token IDs drawn from a fixed vocabulary. Language models as one of the first forward pass steps consume sequence of token IDs as input and map them to their embeddings vectors. A good scheme balances coverage (few unknowns), compact sequences (shorter length for a given text), and stability of encoding/decoding.

2.5.1. Subword methods

Pure word-level vocabularies suffer from out-of-vocabulary (OOV) issues, while character-level tokenization produces very long sequences. Subword tokenization offers a compromise by decomposing words into frequent subunits that compose to any string, including rare words and misspellings [8].

Common approaches:

– **Byte-Pair Encoding (BPE)**: the most widely used method in modern LLMs, starts from a large text corpus and iteratively merges the most frequent adjacent pairs to form larger subwords until a target vocabulary size is reached [8]. BPE yields deterministic segmentations given the learned merges and has no explicit probabilistic model.

– **WordPiece**: subword units to maximize likelihood of a training corpus, typically under a simple probabilistic model over segmentations; greedy decoding selects the best segmentation. It typically includes reserved continuation markers to indicate that a subword is not word-initial.

– **SentencePiece**: optimizes a probabilistic unigram model over a candidate subword inventory and prunes units that hurt likelihood [7]. It supports sampling alternative segmentations during training for robustness and can operate directly on raw text without external pre-tokenization.

Implementation frameworks such as SentencePiece and Transformers provide standardized training, encoding, and decoding pipelines [7, 6]. Variants like byte-level BPE ensure coverage of arbitrary Unicode by operating on UTF-8 bytes, which removes true OOVs at the cost of sometimes longer sequences for non-Latin scripts.

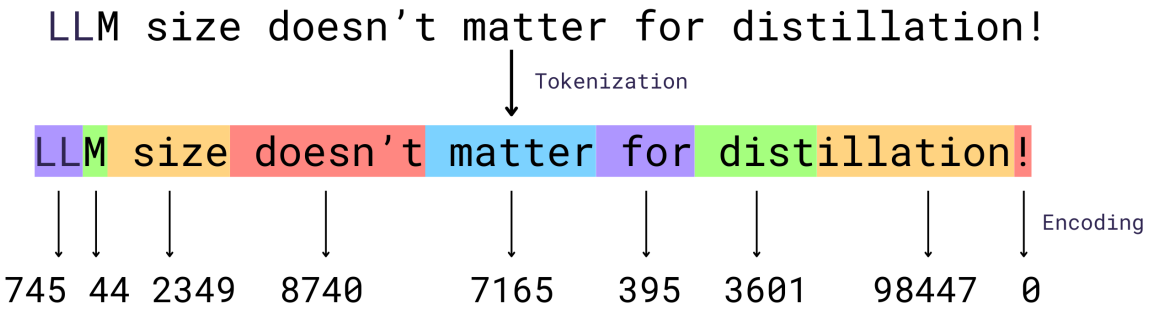


Figure 2.4. Tokenization example for OpenAI GPT-4o Tokenizer.

2.5.1.1. Vocabulary

Larger token vocabularies can represent up to whole words, reducing sequence length and attention cost. However, they implicitly increase embedding and output layer sizes, memory, and may cause overfitting. Smaller vocabularies generalize better to rare forms but inflate sequence length, which increases quadratic attention cost and risk of truncation. Practical choices range from 16k-200k subwords.

2.5.1.2. Normalization and pre-tokenization.

Many pipelines normalize text (e.g., Unicode NFKC, lowercasing, whitespace handling) to improve robustness; SentencePiece can integrate normalization and avoid external word boundary rules [7]. Pre-tokenizers may split on whitespace and punctuation to enforce word boundary awareness before applying subword merges. Choices here affect how punctuation, casing, and whitespace survive round-trip detokenization.

2.5.2. Special tokens and sequence framing

Special tokens mark structure and control decoding:

– **BOS/EOS**: Begin-of-sequence and end-of-sequence delimit the sequence and signal termination.

– **PAD**: Padding aligns sequences within a batch to a common length; attention masks ensure pads are ignored during loss computation and attention.

– **Instruction tokens**: Special markers that indicate task boundaries or specify desired behavior (e.g., `<|user|>`, `<|assistant|>`, `<|thought|>`).

```
<|bos|>
<|user|>
You are given a list of numbers: [3, 1, 4, 1, 5].
Please compute their sum and provide the result.
<|assistant|>
The sum of [3, 1, 4, 1, 5] is 14.
<|eos|>
```

Figure 2.5. Example of conversation-formatted instruction-response prompt.

2.6. GSM8K

GSM8K is a benchmark of 8.5K English grade-school math word problems designed to evaluate multi-step reasoning in natural language [1]. Problems typically require 2-8 steps of elementary arithmetic ($+, -, \times, \div$) and include human-written, natural-language solutions with inline "calculator annotations" (intermediate computations) and a final integer answer on a separate line. The benchmark is widely used to study prompting and reasoning supervision; standard scoring is exact numeric match, often evaluated under chain-of-thought prompting and self-consistency sampling [2, 3].

2.6.1. Dataset structure

Released in English via HuggingFace Datasets [27], GSM8K provides two configurations: a main set (question, rationale, final answer) and a socratic variant that interleaves sub-questions within the solution. Official splits comprise 7,473 training and 1,319 test problems [1]; each instance pairs a question string with a multi-step natural-language solution containing intermediate calculations and a single integer final answer.

Problem

```
Natalia sold clips to 48 of her friends in April, and then she sold half as
many clips in May. How many clips did Natalia sell altogether in April and
May?
```

Solution

```
Natalia sold 48/2 = <<48/2=24>>24 clips in May. Natalia sold 48+24 =
<<48+24=72>>72 clips altogether in April and May. #### 72
```

Figure 2.6. GSM8K example: question, rationale with intermediate steps, and final answer.

3. Methodology

3.1. System requirements

3.1.1. Functional requirements

– Generate a sufficiently large, high-quality, correctly answered set of teacher CoT samples for a GSM8K training subset.

– Persist per-sample metadata (latency, token counts, correctness, cost) for analysis.

– Fine-tune a single small student model to emit coherent step-by-step reasoning and a correctly formatted final answer line.

– Fine-tune a single small student model to emit only the correctly formatted final answer line.

– Evaluate all models (base, students, teacher) on the GSM8K test split under strict numeric exact-match parsing of the final answer line.

3.1.2. Non-functional requirements

– **Optimized scraping**: Highly asynchronous dataset generator with concurrency tuned to balance throughput and timeout/error rate.

– **Observability**: Persistent logging of token usage, latency distributions, and derived cost metrics during dataset generation.

– **Data quality**: Diverse and coherent chain-of-thought reasoning. Removal of outliers.

– **Resource efficiency**: All training and benchmarking runs fit in available VRAM without OOM.

– **Format compliance**: Final answer line must follow “Final Answer: <integer>” exactly. Deviation counts as failure.

– **Maintainability**: Separation of data generation, cleaning, training, and evaluation stages with clear artifact boundaries.

– **Reproducibility**: Deterministic training pipeline, versioned artifacts, prompts, configuration, environment and scripts.

– **Budget adherence**: Monitor spend so teacher generation and GPU usage stay within the stated cap.

3.1.3. Constraints

– Single-GPU environment - no multi-node or distributed training.

– Fixed public benchmark - GSM8K.

– Teacher accessed only via external API.

– Limited sequence context - excessive rationales must be pruned or truncated without losing the final answer line.

– Budget cap at 200PLN.

3.1.4. Success criteria

– Accuracy improvement of the distilled student over the base CoT-prompted model by at least 5 percentage points on GSM8K test.

– Format error rate (cases where numeric answer is correct but formatting invalid) kept below a small threshold (target <5% of all generated correct numbers).

– Cleaned dataset size sufficient to enable measurable gains (tens of thousands of rationale+answer pairs) without exceeding resource limits.

– All runs complete within available GPU memory and budget envelope.

3.1.5. Risks and mitigation

– **Overfitting to limited subset**: Mitigate via data augmentation and training regularization.

– **Format consistency**: Enforce prompt templates and validation during evaluation.

– **Data noise leakage**: Strict filtering to remove incorrect teacher answers before student training.

– **Cost escalation**: Concurrency and request budgeting monitored. Setting account balance limits for GPU usage.

3.2. Architecture

The system is an offline teacher-to-student pipeline that produces a cleaned CoT dataset and trains a 3B-class student with PEFT, then evaluates under strict numeric exact match.

3.2.1. Components

- **Teacher generator**: Async OpenRouter client producing structured CoTs.
- **Prompts/config**: Versioned templates under `prompts/` with a meta JSON capturing decoding + generation config.
- **Dataset artifacts**: Raw, cleaned, and processed JSONL stages (introduced once in Data Flow).
- **Cleaning & preprocessing**: Notebooks applying correctness filtering, rationale dedup, and length outlier removal.
- **Training**: Notebook performing QLoRA fine-tuning of two modes (SCoTD and label-only).
- **Evaluation**: Notebook for greedy exact-match benchmarking.
- **Runtime**: Single Python environment.

3.2.2. Data flow

1. **Ingest**: Load GSM8K train/test splits; select first ∼1,000 train questions.
2. **Generate**: Oversample ∼30 CoTs per question asynchronously and write raw JSONL.
3. **Clean**: Apply correctness filter, rationale dedup, and 99th percentile length trimming and produce cleaned JSONL.
4. **Process**: Project cleaned dataset to minimal supervised learning schema and produce processed JSONL.
5. **Train**: Two modes (SCoTD, label-only).
6. **Evaluate**: Strict numeric exact-match benchmarking.

3.3. Teacher CoT dataset generation

This stage produces a cleaned high-quality rationale+answer corpus from a strong reasoning model.

3.3.1. Subset selection

For cost control only the first ∼1,000 GSM8K train questions are used. Each question targets 30 samples (oversampling for rationale diversity). Generation is resumable: existing `(question_id, sample_id)` pairs found in the output JSONL are skipped on subsequent runs.

3.3.2. Async generation

An async Python client (HTTPX) calls the OpenRouter `/chat/completions` endpoint for model `deepseek/deepseek-r1-distill-qwen-32b`. Concurrency is capped at 25 via an `asyncio.Semaphore`. Each request includes:

- System prompt: concise reasoning format specification.
- User prompt: templated few-shot exemplars + target question.
- Decoding: `temperature=1.0`, `top_p=1.0`.
- Timeout: 120s.

Tenacity applies exponential backoff (max 5 attempts) on transport errors.

3.3.3. Formatting contract

Required output:

```
Reasoning:
- step 1
- step 2
...
Final Answer: <number>
```

No extra trailing text. final line must start exactly with `Final Answer:`. Parser scans lines to extract the numeric suffix.

3.3.4. Parsing and correctness

- Gold answer number: extracted from GSM8K `answer` field after the delimiter using a final-number regex.
- Teacher answer number: last numeric token on the `Final Answer:` line (regex supports integers, decimals, scientific notation).
- Correctness: strict numeric equality (Python int/float comparison). Any parsing failure or missing final line marks the sample invalid (counted as failed).

3.3.5. Recording schema

Each raw sample appended as a JSONL record with fields: `question_id`, `sample_id`, `question`, `gold_answer_text`, `gold_answer_number`, `teacher_answer_text`, `teacher_answer_number`, `is_correct`, `finish_reason`, `usage{prompt_tokens, completion_tokens}`, `latency_ms`, `model`, `request_id`. Latency is wall-clock from request dispatch to response receipt.

3.3.6. Resumability

Before generating new samples, existing JSONL is scanned to build a per-question set of occupied `sample_id`s. Missing IDs in $[0, N)$ are generated where completed questions are skipped entirely.

3.3.7. Progress and metrics

A live tqdm progress bar displays:

- **cost**: computed continuously as
$$\text{cost} = \frac{T_{in}}{10^6} c_{in} + \frac{T_{out}}{10^6} c_{out}$$
with $c_{in} = c_{out} = \$0.27$/M tokens (taken from OpenRouter).
- **tokens_in / tokens_out**: cumulative prompt / completion tokens.
- **acc**: running fraction of `correct_samples / total_samples`.
- **ok/err**: successful vs failed attempts.

3.3.8. Failure handling

Failures include HTTP errors, timeouts, parsing errors, or malformed responses. Each increments `failed_samples` and no partial record is written. Retries only cover transport exceptions—logical format violations are not retried to avoid cost inflation.

3.3.9. Quality outcome

Oversampling plus strict filtering yields a pool of diverse, correctly formatted rationales per question (typically multiple distinct reasoning paths). This supports effective downstream distillation while keeping total spend within the budget envelope.

3.4. Cleaning

This stage removes incorrect, duplicate, and extreme-length samples to yield a high-signal supervision set.

3.4.1. Input

Raw JSONL: `artifacts/data/raw/dataset.jsonl` containing 30,000 generations (1,000 questions $\times$ 30 samples), each with correctness flag, usage tokens, and rationale text.

3.4.2. Operations

1. **Load & index**: Read JSONL into a DataFrame indexed by `(question_id, sample_id)` for deterministic ordering.

2. **Correctness filter**: Keep only rows where `is_correct = True`. Removes all teacher numeric mismatches; prevents label noise (primary gain).

3. **Exact rationale dedup**: Drop duplicates on `teacher_answer_text`. Prevents overweighting identical reasoning paths.

4. **Token-length outlier removal**: Compute per-row `total_tokens = prompt_tokens + completion_tokens`. Remove rows with `total_tokens` $\geq$ 99th percentile (threshold $\approx$1392 tokens). This trims extreme verbose rationales that inflate sequence length and memory without adding diversity.

5. **Persist**: Reset index and write cleaned JSONL to `artifacts/data/cleaned/dataset.jsonl`.

3.4.3. Justification

– **Strict correctness**: Ensures the student never trains on wrong final answers (no noisy label memorization).

– **Deduplication**: Reduces redundant gradient steps; improves effective rationale variety per batch.

– **Length capping**: Controls worst-case sequence length to avoid GPU OOM and keeps average tokens closer to the median (739 total tokens).

3.4.4. Outcome

Resulting size: 24,952 cleaned samples (from 30k raw). Teacher correctness in retained set is 100%. Distribution tails (> 99th percentile) removed while preserving all final answer lines.

3.5. Preprocessing

Transforms the cleaned corpus into the minimal supervision schema consumed by training scripts.

3.5.1. Input

Cleaned JSONL: `artifacts/data/cleaned/dataset.jsonl` (indexed by `question_id`, `sample_id`).

3.5.2. Output

Select only: `question`, `gold_answer_text`, `gold_answer_number`, `teacher_answer_text` (rationale + final answer line), `teacher_answer_number` Index columns are preserved until final write, then flattened.

3.5.3. Artifact

Write to `artifacts/data/processed/dataset.jsonl`.

3.5.4. Schema Rationale

Minimal fields reduce I/O and memory footprint, remove unused metadata (latency, token usage) for training, and keep both textual and numeric answers to enable strict evaluation parsing and auxiliary checks.

3.5.5. Integrity

All records already passed strict correctness; no further validation required. Numeric fields (`gold_answer_number`, `teacher_answer_number`) support exact-match evaluation; textual rationale preserved verbatim for supervised reasoning.

3.6. Prompt templates

This section enumerates the fixed prompt templates used across generation, training, and benchmarking. Templates are versioned as plain text under `code/prompts/`. No runtime mutation other than `{question}` placeholder interpolation.

3.6.1. Files and roles

– **CoT system** (`code/prompts/cot/system.txt`): Instructs concise multi-step reasoning (2–8 steps) plus a terminating final answer line.

– **CoT user** (`code/prompts/cot/user.txt`): Provides three few-shot exemplars (question, reasoning steps, final answer) followed by the target question placeholder.

– **Label-only system** (`code/prompts/label_only/system.txt`): Constrains output to a single final answer line with no reasoning.

– **Label-only user** (`code/prompts/label_only/user.txt`): Supplies same three few-shot answer-only exemplars plus the target question placeholder.

3.6.2. Formatting contracts

– **CoT mode**:

```
Reasoning:
- step 1
- step 2
...
```

`Final Answer: <number>`

2–8 dash-prefixed steps; arithmetic shown explicitly; no trailing text after the final answer line.

– **Label-only mode**: Single line `Final Answer: <number>` No units, commentary, or extra lines.

3.7. Training

Goal of training is to fine-tune a single 3B decoder-only model (Qwen/Qwen2.5-3B) with QLoRA under 4-bit NF4 quantization and LoRA adapters, using bf16 compute and greedy decoding for evaluation.

3.7.1. Model, quantization, and PEFT

– Base: `Qwen/Qwen2.5-3B` loaded with BitsAndBytes 4-bit (`nf4`, double-quantization), compute dtype bf16; `device_map="auto"`.

– Gradient checkpointing enabled; `model.config.use_cache = false` during training.

– Tokenizer: fast; if `pad_token` is missing, set to `eos_token` and propagate pad IDs to model and generation config.

– LoRA: `r=16`, `alpha=32`, `dropout=0.05`, `bias="none"` on {`q_proj`, `k_proj`, `v_proj`, `o_proj`, `gate_proj`, `up_proj`, `down_proj`}; task type `CAUSAL_LM`.

3.7.2. Data split and reproducibility

– Input: `artifacts/data/processed/dataset.jsonl`.

– Split: 80/20 by `question_id` (`TRAIN_SPLIT=0.8`) with `seed=42` ensures no question leakage across splits.

– Label-only mode dedup: keep a single record per `question_id` (first by `sample_id`) to avoid multiple labels per question.

– Seeds for Python, PyTorch, CUDA, and accelerate set to 42; TF32 enabled on matmul/CuDNN.

3.7.3. Prompting and target formatting

– Prompts: training/inference use the same templates.

  – SCoTD: `prompts/cot/system.txt` + `prompts/cot/user.txt`.

  - Label-only: `prompts/label_only/system.txt` + `prompts/label_only/user.txt`.

- Targets:

  - SCoTD: full teacher rationale text including the final line.
  - Label-only: a single line "Final Answer: <number>" built from the gold number.

3.7.4. Sequence construction and truncation

- Max sequence length: 2048.
- Encoding: concatenate prompt IDs with answer IDs; only answer tokens contribute to loss.
- Truncation policy: always preserve the full answer. If needed, trim the prompt from the left so that $|\text{prompt}| + |\text{answer}| \leq 2048$.
- Loss mask: labels are `-100` for all prompt positions and the token IDs of the answer (incl. EOS) for answer positions.

3.7.5. Batching and collator

- Dynamic padding to the longest in batch; pad to a multiple of 8 for Tensor Cores; attention mask computed as `input_ids != pad_id`.
- `group_by_length=true` to reduce padding waste.

3.7.6. Optimization and schedule

- Trainer: HuggingFace `Trainer`.
- Optimizer: `paged_adamw_8bit`.
- Scheduler: linear; `warmup_ratio` per mode (below).
- Precision: bf16 if hardware supports it, fp16 otherwise.
- Dataloading: `num_workers=4`, `pin_memory=true`.
- Checkpointing: `save_strategy="steps"`; `load_best_model_at_end=true` with `metric_for_best_model="eval_loss"` and `greater_is_better=false`.
- Logging: TensorBoard, `logging_steps=10`.

3.7.7. Mode-specific hyperparameters

SCoTD (rationale+answer):

- epochs=10, learning rate=2×10^{-5}.
- per-device train/eval batch size=8, grad accumulation=2.
- warmup_ratio=0.03, weight_decay=0.
- save/eval every 200 steps.

Label-only (answer line only):

- epochs=20, learning rate=1×10^{-5}.
- per-device train/eval batch size=16, grad accumulation=1.
- warmup_ratio=0.1, weight_decay=0.1, max_grad_norm=1.0.
- save/eval every 50 steps.

3.7.8. Export and sanity check

- After training, reload the frozen 4-bit base and attach the best LoRA checkpoint (`PeftModel.from_pretrained`) for inference.
- Deterministic, greedy generation (`do_sample=false`) used for quick sanity checks on held-out questions; decoding respects set `pad_token_id` and `eos_token_id`.

3.8. Benchmarking

Benchmarking evaluates the base model and all student checkpoints on the GSM8K test split under strict exact-match parsing of the final numeric answer.

3.8.1. Objective

Measure accuracy ($\#correct \div \#total$) for:

- Base model with CoT prompting.
- Base model with label-only prompting.
- All SCoTD LoRA checkpoints.
- All label-only LoRA checkpoints.

All generations use deterministic greedy decoding (no sampling, no self-consistency).

3.8.2. Dataset and gold answers

The GSM8K test split is loaded via `load_dataset(GSM8K_PATH, name="main", split="test")`. Gold numeric answers are extracted from the original `answer` field using `parse_gold_answer_number`, which regex-parses the final integer after "####" delimiter.

3.8.3. Prompt construction

Two builders create inference prompts:

- `build_prompt_cot`: concatenates the CoT system template and user template with the target question; expects rationale steps then a final line.
- `build_prompt_label_only`: analogous, but the system template instructs answer-only output.

Both yield a single plain-text prompt string; any truncation (left side) preserves the answer region.

3.8.4. Model loading and quantization

Each run loads either the base model (`Qwen/Qwen2.5-3B`) or a LoRA checkpoint path. All models use 4-bit NF4 quantization (`BitsAndBytesConfig` with double quantization) and bf16 compute. `device_map="auto"`. `model.config.use_cache = True` activates KV caching for faster greedy decoding.

3.8.5. Tokenizer settings

Tokenizer is fast, left-padding and left-truncating. Pad token ID is propagated to `generation_config`. Batches are tokenized with dynamic padding; no special postprocessing beyond stripping decoded prompt prefixes from responses.

3.8.6. Generation configuration

- Greedy: `do_sample=False`.
- `max_new_tokens=1024` to allow long rationales.
- Cache usage: `use_cache=True`.
- Precision aids: TF32 enabled on matmul/CuDNN for potential minor throughput gains.

3.8.7. Batching

Batch size reflects output length expectations:

- Label-only: 128 (short single-line answers).

– CoT: 16 (long multi-step rationales).

Processing iterates over slices of question lists; each batch calls `model.generate` once.

3.8.8. Parsing and strict evaluation

Predicted texts are trimmed by removing the prompt prefix and then parsed with `parse_teacher_final_answer`, which searches for a final line starting with "Final Answer:" and extracts the trailing numeric token. Failures (missing line, malformed number, regex miss) raise `ParsingError`; such cases count as incorrect. This strictness penalizes:

– Format deviations (e.g., extra commentary after the answer line).

– Missing prefix "Final Answer:".

– Multiple numeric candidates (the parser only accepts the final numeric token on that line).

Accuracy is the fraction of predictions whose parsed number equals the gold number (exact equality). No normalization (e.g., stripping commas or units) is applied.

3.8.9. Checkpoint iteration and resource cleanup

A `RUNS` list enumerates all label-only checkpoints (sorted by numeric suffix), all SCoTD checkpoints, then the two base-mode baselines. After each run:

– Metrics (accuracy, count) are appended to a results list.

– Predictions DataFrame rows receive the model name.

– `cleanup_model` deletes model/tokenizer objects, triggers garbage collection, empties CUDA cache, and synchronizes to reduce fragmentation before loading the next checkpoint.

This prevents cumulative GPU memory growth across many sequential loads.

3.8.10. Artifacts

All per-question predictions across runs are concatenated and written to `predictions.csv` with columns:

– `model` (run identifier)

– `question`

– `prediction` (raw decoded answer text)

– `gold_answer` (numeric gold)

3.9. Environment setup

Fine-tuning and benchmarking runs were executed on RunPod node with the following specs:

3.9.1. Hardware

– Single GPU: RunPod RTX 4090 (24GB VRAM), host RAM 64GB.

– Pod cost: $0.34/hour.

3.9.2. Software

– Python 3.11; package manager: `uv` (`uv.lock` committed).

– CUDA 12.9; PyTorch 2.8.0.

– Env: `TOKENIZERS_PARALLELISM=false`.

4. Results

This chapter reports empirical outcomes of supervised chain-of-thought distillation (SCoTD) and label-only finetuning on GSM8K. All metrics use strict numeric exact-match evaluation under greedy decoding. No self-consistency sampling or answer normalization was applied.

4.1. Evaluation setup recap

- Base model: `Qwen/Qwen2.5-3B`, no finetuning.
- Students: LoRA adapters trained on cleaned teacher data (SCoTD: rationale+answer, Label-only: final answer line only).
- Teacher: `deepseek/deepseek-r1-distill-qwen-32b` via API.
- Decoding: greedy (`do_sample=false`) for all benchmarks.
- Metric: accuracy = correct numeric parses / total questions (1,319 GSM8K test problems).
- Parsing: requires a line starting `Final Answer:` followed by a single numeric token; any deviation counts as incorrect.

4.2. Headline accuracies

- Teacher (API): 93.61%.
- Student SCoTD (best checkpoint): 78.54%.
- Base (CoT prompting, no finetune): 70.51%.
- Student label-only (best checkpoint): 15.01%.
- Base (label-only prompting): 10.99%.

Thus SCoTD yields a +8.03 percentage-point gain over the base CoT baseline, while label-only supervision gives a +4.02 point gain over the label-only baseline under strict formatting.

4.3. Checkpoint dynamics

4.3.1. SCoTD

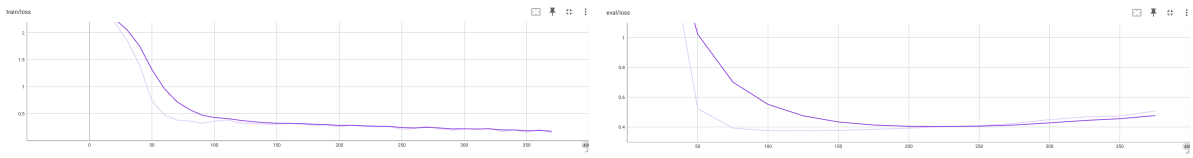
Model saved every 200 training steps. Best accuracy observed at step 1,400 (78.54%). Checkpoints after this point exhibit slight stagnation (no further gains) despite continued training loss decrease, indicating diminishing returns and potential mild overfitting to rationale style rather than numeric robustness.

4.3.2. Label-only

Saved every 50 steps; best accuracy at step 200 (15.01%). Subsequent checkpoints degrade or plateau, correlating with rising validation loss after early minimum (Figure: label-only eval loss curve). Early stopping at the first validation-loss inflection would preserve best generalization.

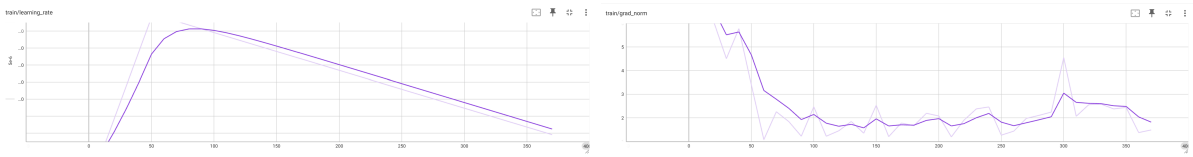
4.4. Training behavior

4.4.1. Label-only mode (answer line only)



(a) Train loss (b) Eval loss

Figure 4.1. Label-only losses: rapid early train loss compression; eval loss reaches minimum near 180–220 steps then rises (overfitting onset).



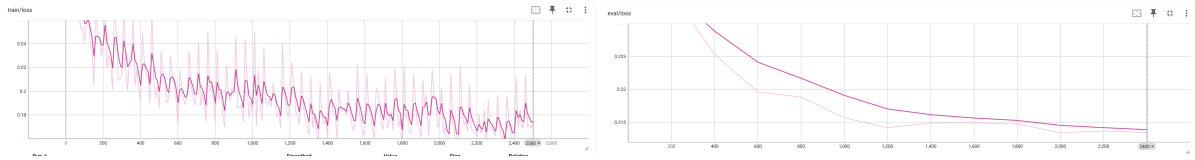
(a) Learning rate (b) Grad norm

Figure 4.2. Label-only optimization dynamics: warmup then linear LR decay; early high gradient norms (>6) collapse to a stable 1.5–2.5 band.

Observations:

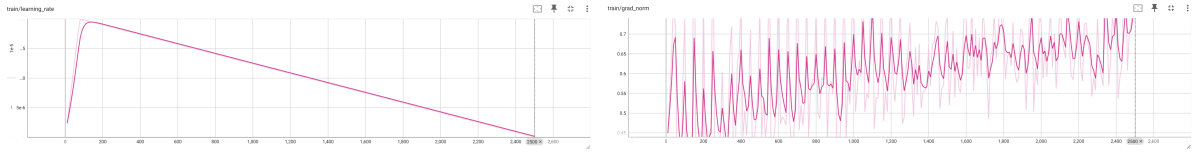
- Rapid optimization: train loss drops >75% within first $\tilde{1}$20 steps (Figure **??**a).
- Generalization peak: eval loss minimum near 180–220 steps (Figure **??**b) aligns with best accuracy (15.01%); subsequent rise indicates overfitting on formatting/token patterns.
- Gradient norm collapse: large initial adaptation (Figure **??**b) precedes eval loss floor; post-collapse stability reflects limited sequence length.

4.4.2. SCoTD mode (rationale + answer)



(a) Train loss (b) Eval loss

Figure 4.3. SCoTD losses: steady train loss descent; eval loss monotonically declines without early reversal.



(a) Learning rate (b) Grad norm

Figure 4.4. SCoTD dynamics: short warmup then linear LR decay; grad norms in a tight 0.45–0.75 band with slight late drift upward.

Observations:

– Stable optimization: absence of large spikes (Figure **??**b) due to longer sequences distributing updates.

– Continued eval improvement: eval loss still falling past best accuracy checkpoint (Figure **??**b) implying accuracy saturation before token-level optimum.

– Low variance: constrained grad norm range suggests consistent convergence; rationale diversity did not destabilize training.

4.4.3. Comparison

– Label-only: faster early loss reduction (Figure **??**a) but earlier generalization peak and overfitting (Figure **??**b).

– SCoTD: sustained eval loss improvement (Figure **??**b); accuracy plateau likely reflects reasoning ceiling rather than overfitting.

– Gradient behavior: high initial label-only norms (Figure **??**b) vs. SCoTD's moderate steady norms (Figure **??**b) indicate incremental adaptation with longer rationales.

4.5. Output length and latency distributions

Quantitative summaries:

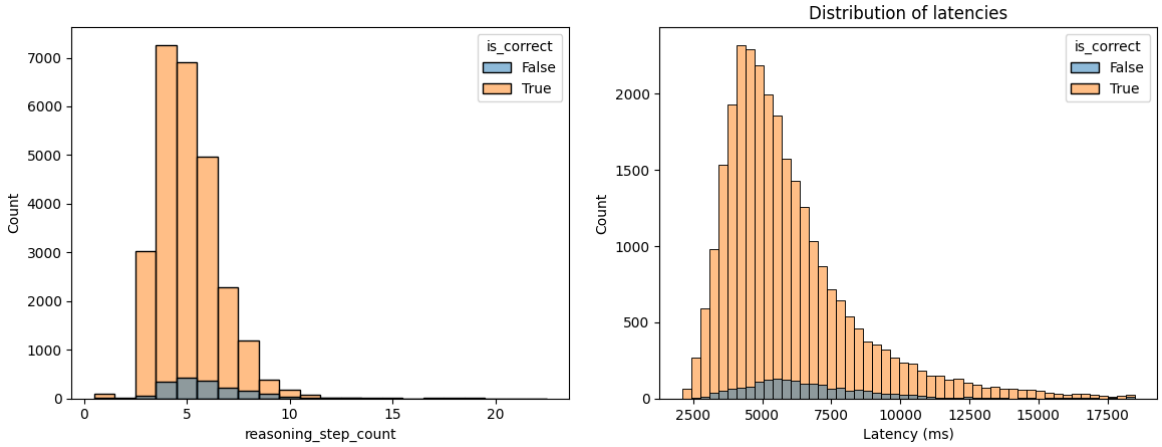


Figure 4.5. Left: reasoning step count (mode 5, most in [4,7], rare long tails >12). Right: latency distribution (median $\tilde{5}$.4s, long right tail to >18s).

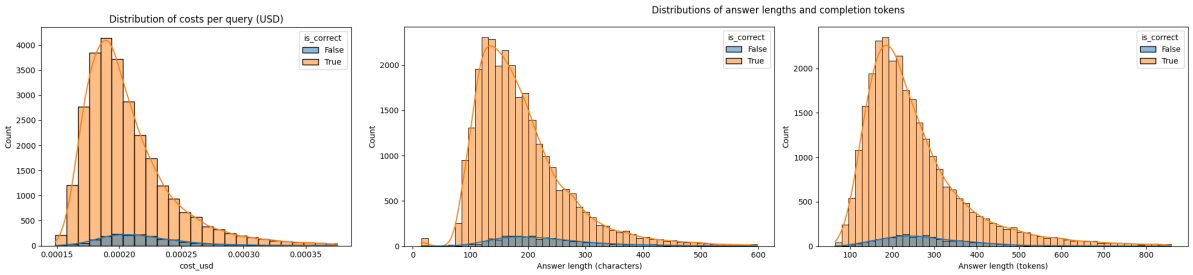


Figure 4.6. Left: cost per query (right-skew, peak near \$0.00019–0.00021). Right: answer length distributions (characters and tokens; token mode $\tilde{1}$80, tail >800).

– Step count: 90% within 4–7 steps; incorrect samples slightly over-represented at 4–5 steps (shorter reasoning).

– Latency: interquartile band concentrated between $\tilde{4}$.3s and $\tilde{7}$.1s; heavy tail aligns with longest completions.

– Cost: distribution approximates log-normal; long tail beyond \$0.00030 mainly verbose rationales.

– Length: character median $\tilde{1}$50; token median 225 (matches earlier quantiles); incorrect answers mildly skew shorter (information insufficiency).

4.6. Dataset filtering outcomes

4.6.1. Raw and cleaned sizes

– Raw generated samples: 30,000 (1,000 train questions $\times$ 30 teacher generations).

– Teacher overall numeric correctness (raw): 93.75% (28,125 correct).

– Cleaned samples retained: 24,952 (100% correct after filtering).

4.6.2. Removal breakdown (approximate)

– Incorrect answers removed: 1,875.

– Length outliers (> 99th percentile total tokens; threshold ≈1,392): ∼300 (at most 1%).

– Exact duplicate rationales removed: ∼2,873 (difference after accounting for outliers).

Note: Duplicate vs outlier counts are approximate; precise tallies depend on overlapping removal cases.

4.6.3. Effect

Filtering ensures zero label noise and improved rationale diversity while capping worst-case sequence length for memory stability.

4.7. Cost analysis

4.7.1. Aggregate

– Total teacher generation cost: $6.33240.

– Average cost per raw query: $0.00021 (median $0.00020; std $0.000062).

– Cost per retained cleaned sample: $6.33240 / 24,952 ≈ $0.000254.

4.7.2. Distribution

Histogram shows a right-skew with a long tail up to ∼$0.00037 (Figure: cost per query). Tail samples correspond to longer completions (higher completion token counts). Tight clustering around $0.00020 indicates stable prompt length and moderate variance in rationale length.

5. Discussion

5.1. Interpretation of quantitative outcomes

The distilled student (78.54% accuracy) improves over the base CoT-prompted model (70.51%) by +8.03 percentage points under strict greedy evaluation. This gain indicates that supervised chain-of-thought distillation (SCoTD) successfully transfers structured multi-step reasoning patterns rather than merely formatting habits. The teacher at 93.61% establishes remaining headroom: even after distillation the student closes only about 35% of the gap to the teacher (absolute difference 23.10 points). Label-only fine-tuning produces a smaller +4.02 point improvement (15.01% vs 10.99%), reflecting that (i) training only the final answer line provides limited signal for intermediate decomposition, and (ii) strict formatting magnifies penalties for subtle deviations.

5.2. Role of dataset filtering

The cleaned dataset (24,952 samples, 100% numerically correct) removes incorrect answers, near-duplicate rationales, and extreme-length outliers. Eliminating label noise prevents the student from learning wrong numeric terminations; rationale deduplication reduces gradient redundancy and broadens reasoning variety; length capping protects memory while retaining core arithmetic trajectories. The monotonic eval loss decline in SCoTD training suggests the filtering did not over-prune diversity needed for generalization.

5.3. Training dynamics and generalization

Label-only mode shows rapid early optimization, an early eval loss floor, and overfitting signs after $\tilde{2}00$ steps, consistent with short targets and limited supervision richness. SCoTD exhibits slower but steadier improvement: eval loss continues decreasing past the best accuracy checkpoint, implying a saturation of accuracy before full convergence of token-level modeling. This mismatch indicates that incremental gains in reproducing teacher phrasing beyond a certain point do not translate into additional numeric correctness under greedy decoding.

5.4. Formatting sensitivity

Strict parsing (exact “Final Answer: <number>”) penalizes:

– Additional commentary after the numeric token.

– Missing prefix or variant capitalization.

– Multiple numeric candidates on the line.

The observed low label-only accuracy relative to intuitive difficulty of many questions suggests a non-trivial fraction of outputs may contain correct numbers with minor formatting deviations (not separately quantified—future work should measure the formatting error rate). This evaluation design intentionally preserves a hard constraint to highlight robustness rather than lenient post-processing.

5.5. Cost and efficiency implications

Teacher generation cost $6.33 produced a high-signal corpus enabling a one-time accuracy gain that permanently reduces inference-time API spend. The test-set teacher cost estimate ($0.282) contrasts with the student’s negligible marginal cost (local greedy decode). Thus, SCoTD shifts expenditure forward (data collection + training) to amortize savings over repeated evaluations. Oversampling (30 CoTs/question) proved feasible and economically justified given low per-sample cost ($0.00021 mean).

5.6. Limitations

– Single dataset (GSM8K) restricts external validity; reasoning domains with more symbolic complexity (algebraic manipulation, formal proofs) may demand different supervision properties.

– Single backbone (Qwen2.5-3B) prevents assessing scale effect or architectural sensitivity (e.g., mixture-of-experts, higher-rank attention).

– No exploration of self-consistency decoding [3] at inference; potential additive gains remain unmeasured.

– No ablation on filtering stages (e.g., retaining some incorrect but diverse rationales) to quantify trade-off between purity and diversity.

– Formatting error rate not isolated; lack of tolerant or multi-metric evaluation (e.g., relaxed number matching, rationale quality scoring).

5.7. Threats to validity

– **Data contamination**: Unverified overlap of teacher pretraining data with GSM8K could inflate teacher correctness; mitigation: use official test split and public model, but provenance uncertainty remains.

– **Random seed dependence**: Single-seed runs may mask variance in adapter training; confidence intervals absent.

– **Hardware-specific behavior**: 4-bit quantization stability can vary across GPU architectures; results obtained on RTX 4090 may not exactly reproduce on other devices.

– **Parser rigidity**: Strict final-line parser potentially underestimates true numeric capability, especially for label-only.

– **Checkpoint selection bias**: Choosing best eval-loss checkpoint may not maximize accuracy if loss continues decreasing after accuracy plateau; strategy was accuracy-based (step 1400 SCoTD), minimizing this threat but still single-criterion.

5.8. Implications

– High-quality filtered CoTs enable a small model to internalize multi-step arithmetic decomposition without increasing inference latency.

– Parameter-efficient fine-tuning (LoRA + QLoRA) is sufficient to encode reasoning gains on a single commodity GPU, reinforcing PEFT viability for resource-constrained deployments.

– Rationale supervision offers better return than answer-only supervision for math word problems under greedy decoding; intermediate structure is a critical training signal.

– Evaluation protocol design (strict formatting) materially impacts measured accuracy; downstream applications requiring reliability should incorporate format validation loops or structured decoding.

5.9. Comparison to literature

Observed gains align with symbolic CoT distillation reports [21], supporting the importance of multiple rationales per question. Difference from token/logit distillation paradigms [16, 19] is that sequence-level supervised rationale reproduction alone suffices for measurable reasoning improvement at small scale. The pipeline demonstrates integration of QLoRA [5] with multi-step reasoning tasks and confirms memory efficiency without compromising accuracy trajectory.

5.10. Future work

– Quantify formatting vs numeric error decomposition to refine evaluation and potentially apply lightweight post-formatting corrections.

– Introduce self-consistency sampling for the distilled student to assess marginal gains beyond greedy decoding.

– Multi-dataset extension (e.g., SVAMP, AQuA) to evaluate cross-domain generalization of distilled reasoning.

– Ablation of filtering (keep partial incorrect rationales) to examine whether limited noise improves robustness or harms accuracy.

– Hybrid loss with partial token-level KL to teacher logits where available, combining rationale supervision with softened distribution alignment.

– Curriculum ordering (short $\rightarrow$ long rationales) to test whether sequence length progression affects convergence speed or final accuracy.

6. Conclusion

6.1. Summary of objectives

The thesis investigated whether supervised chain-of-thought distillation (SCoTD) can improve a 3B-class small language model's math reasoning accuracy on GSM8K under strict greedy evaluation, and compared rationale+answer supervision to answer-only (label-only) fine-tuning within a constrained single-GPU environment.

6.2. Answers to research questions

RQ1: SCoTD improved the student from 70.51% (base with CoT prompting) to 78.54% (+8.03 points). Thus, supervised rationale transfer is effective for multi-step arithmetic in a 3B model under resource constraints.

RQ2: Label-only fine-tuning improved from 10.99% to 15.01% (+4.02 points) but remained far below SCoTD. Intermediate supervision is substantially more informative than final-answer-only supervision for this task and evaluation regime.

6.3. Key conclusions

– Distilling structured teacher reasoning traces yields significant accuracy gains without increased inference-time cost.

– Rationale diversity (30 generations/question) combined with correctness filtering and deduplication produces a high-signal training set.

– Parameter-efficient fine-tuning (LoRA + QLoRA) provides adequate adaptation capacity for reasoning improvements in small models.

– Strict formatting multiplies penalties for minor output deviations, especially for label-only training; evaluation design materially affects reported performance.

– Accuracy improvement saturates before full token-level convergence; additional training after the best accuracy checkpoint mainly refines stylistic fidelity.

6.4. Practical guidance

- **Data collection**: Oversample teacher CoTs per question, then filter strictly for numeric correctness and deduplicate exact rationale strings.
- **Length control**: Remove extreme-length outliers (>99th percentile total tokens) to avoid memory spikes without harming diversity.
- **Training**: Monitor eval loss and accuracy concurrently; implement early stopping once accuracy plateaus to save compute.
- **Formatting robustness**: Enforce output templates during training; consider lightweight post-generation format validation or constrained decoding for production.
- **Resource efficiency**: Use 4-bit quantization plus LoRA ranks tuned to memory budget; enable gradient checkpointing and bf16 for stability.
- **Evaluation**: Maintain a strict parser for reproducibility; separately track formatting vs numeric errors for diagnostic insight.

6.5. Limitations recap

Single dataset and backbone, absence of self-consistency decoding, no controlled ablations on filtering components, and unquantified formatting error rates limit generalizability and completeness.

6.6. Recommended next steps

Extend to additional reasoning benchmarks, measure formatting error breakdown, experiment with self-consistency for the distilled student, and test hybrid rationale+logit distillation. Evaluate cross-task adapter composability and curriculum sequencing of rationales.

6.7. Closing

Supervised chain-of-thought distillation offers a cost-efficient method to elevate small model reasoning without reliance on large teacher inference at deployment time. Under constrained hardware, combining high-quality rationale curation with parameter-efficient fine-tuning yields meaningful accuracy gains and provides a reproducible path toward more capable small-scale reasoning systems.

Bibliography

[1] Karl Cobbe et al. „*Training Verifiers to Solve Math Word Problems*". 2021. arXiv: *2110.14168* [cs.LG].

[2] Jason Wei et al. „*Chain-of-Thought Prompting Elicits Reasoning in Large Language Models*". 2023. arXiv: *2201.11903* [cs.CL].

[3] Xuezhi Wang et al. „*Self-Consistency Improves Chain of Thought Reasoning in Language Models*". 2023. arXiv: *2203.11171* [cs.CL].

[4] Edward J. Hu et al. „*LoRA: Low-Rank Adaptation of Large Language Models*". 2021. arXiv: *2106.09685* [cs.CL].

[5] Tim Dettmers et al. „*QLoRA: Efficient Finetuning of Quantized LLMs*". 2023. arXiv: *2305.14314* [cs.LG].

[6] Thomas Wolf et al. „*HuggingFace's Transformers: State-of-the-art Natural Language Processing*". 2020. arXiv: *1910.03771* [cs.CL].

[7] Taku Kudo and John Richardson. „*SentencePiece: A simple and language independent subword tokenizer and detokenizer for Neural Text Processing*". 2018. arXiv: *1808.06226* [cs.CL].

[8] Rico Sennrich, Barry Haddow, and Alexandra Birch. „*Neural Machine Translation of Rare Words with Subword Units*". 2016. arXiv: *1508.07909* [cs.CL].

[9] Qwen et al. „*Qwen2.5 Technical Report*". 2025. arXiv: *2412.15115* [cs.CL].

[10] DeepSeek-AI et al. „*DeepSeek-R1: Incentivizing Reasoning Capability in LLMs via Reinforcement Learning*". 2025. arXiv: *2501.12948* [cs.CL].

[11] OpenRouter. „*OpenRouter: The Unified Inference for LLMs*". *https://openrouter.ai*. Accessed 2025. 2023.

[12] Ashish Vaswani et al. „*Attention Is All You Need*". 2023. arXiv: *1706.03762* [cs.CL].

[13] Jared Kaplan et al. „*Scaling Laws for Neural Language Models*". 2020. arXiv: *2001.08361* [cs.LG].

[14] Tom B. Brown et al. „*Language Models are Few-Shot Learners*". 2020. arXiv: *2005.14165* [cs.CL].

[15] Long Ouyang et al. „*Training language models to follow instructions with human feedback*". 2022. arXiv: *2203.02155* [cs.CL].

[16] Geoffrey Hinton, Oriol Vinyals, and Jeff Dean. „*Distilling the Knowledge in a Neural Network*". 2015. arXiv: *1503.02531* [cs.CL].

[17] Tommaso Furlanello et al. „*Born Again Neural Networks*". 2018. arXiv: *1805.04770* [cs.LG].

[18] Qizhe Xie et al. „*Self-training with Noisy Student improves ImageNet classification*". 2020. arXiv: *1911.04252* [cs.LG].

[19] Jianping Gou et al. „*Knowledge Distillation: A Survey*". 2021. arXiv: *2006.05525* [cs.LG].

[20] Yoon Kim and Alexander M. Rush. „*Sequence-Level Knowledge Distillation*". 2016. arXiv: *1606.07947* [cs.CL].

[21] Liunian Harold Li et al. „*Symbolic Chain-of-Thought Distillation: Small Models Can Also "Think" Step-by-Step*". 2024. arXiv: *2306.14050* [cs.CL].

[22] Neil Houlsby et al. „*Parameter-Efficient Transfer Learning for NLP*". 2019. arXiv: *1902.00751* [cs.LG].

[23] Brian Lester, Rami Al-Rfou, and Noah Constant. „*The Power of Scale for Parameter-Efficient Prompt Tuning*". 2021. arXiv: *2104.08691* [cs.CL].

[24] Xiang Lisa Li and Percy Liang. „*Prefix-Tuning: Optimizing Continuous Prompts for Generation*". 2021. arXiv: *2101.00190* [cs.CL].

[25] „*bitsandbytes*". *https://github.com/bitsandbytes-foundation/bitsandbytes*.

[26] NVIDIA Corporation. „*QLoRA Parameter-Efficient Fine-Tuning - NVIDIA NeMo Documentation*". 2024. URL: *https://docs.nvidia.com/nemo-framework/user-guide/24.09/sft_peft/qlora.html*.

[27] Quentin Lhoest et al. „*Datasets: A Community Library for Natural Language Processing*". 2021. arXiv: *2109.02846* [cs.CL].